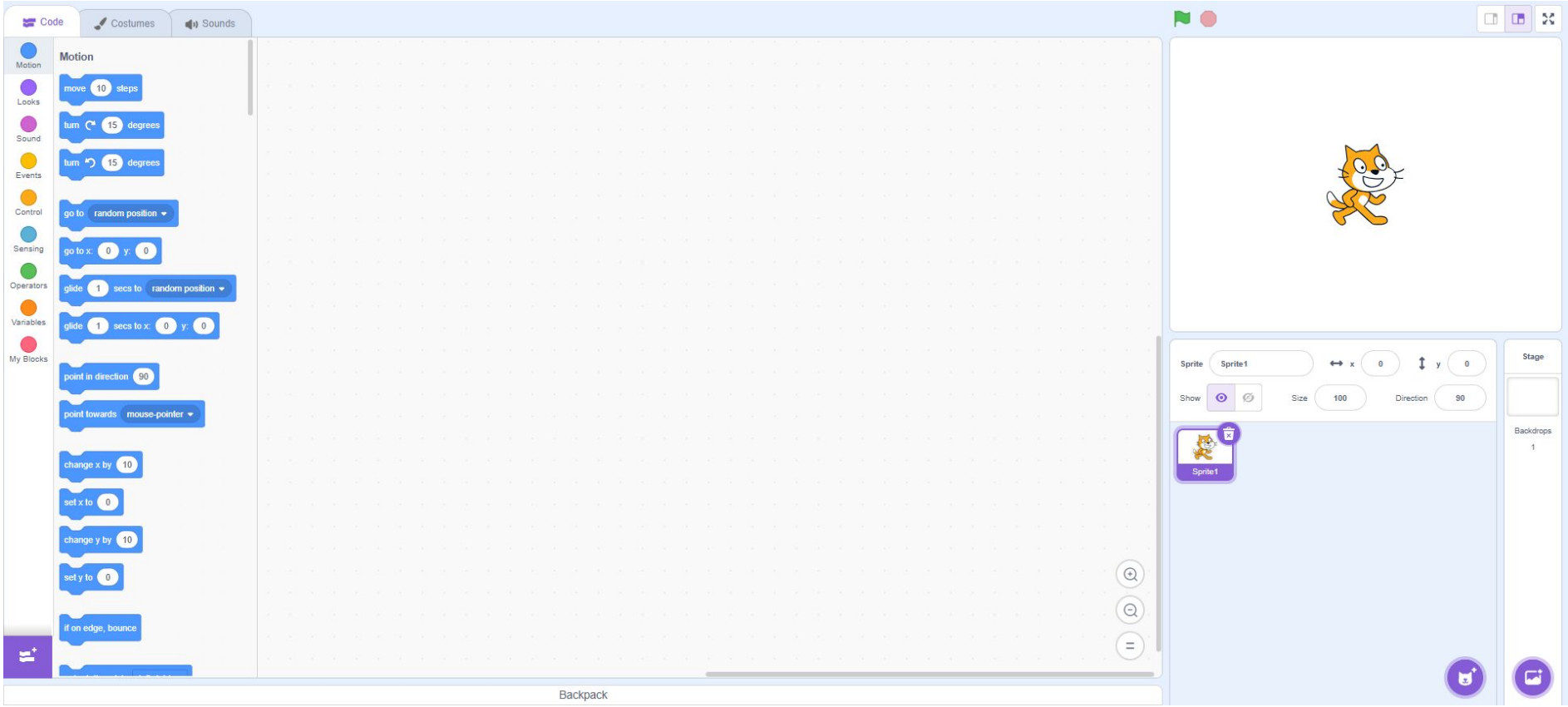
Game Development

With Scratch

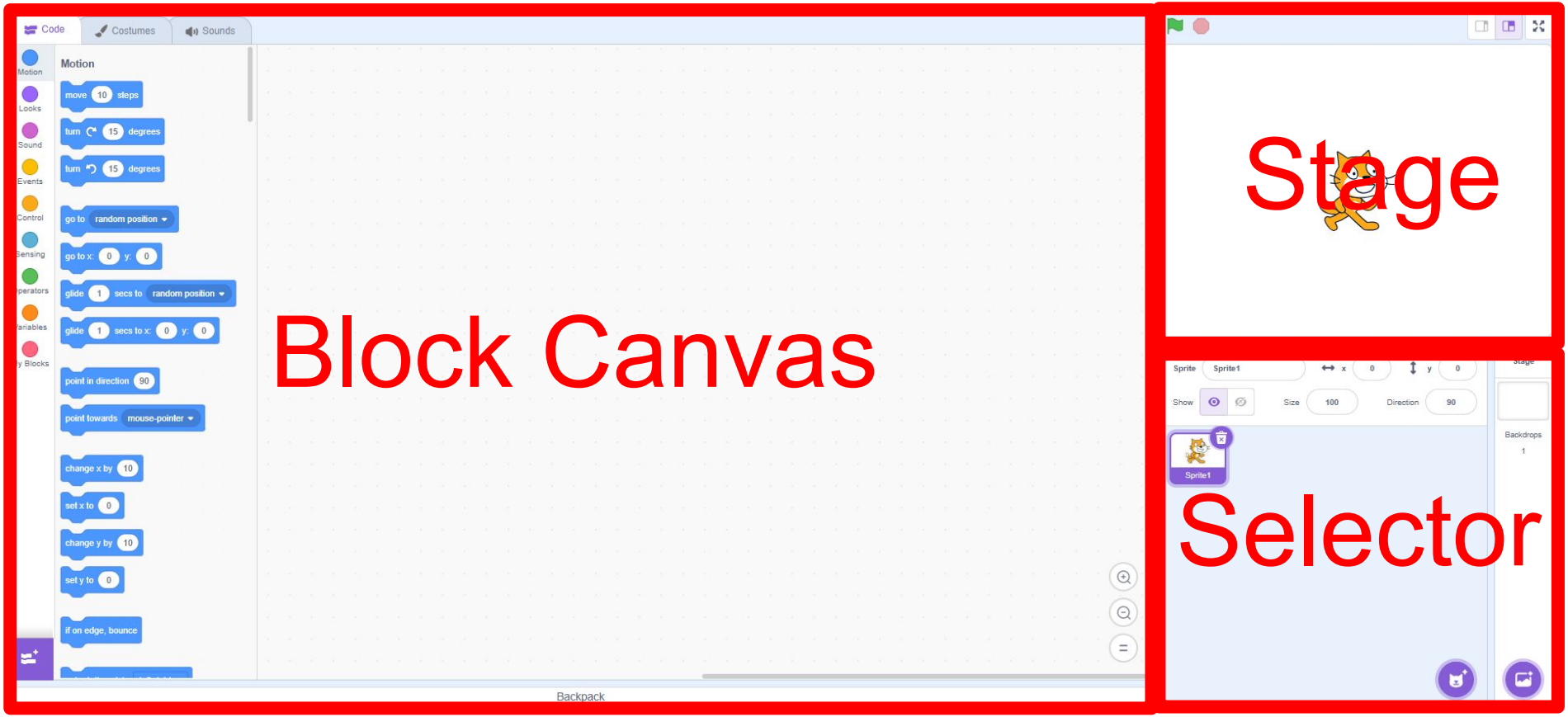
Intro to scratch.mit.edu

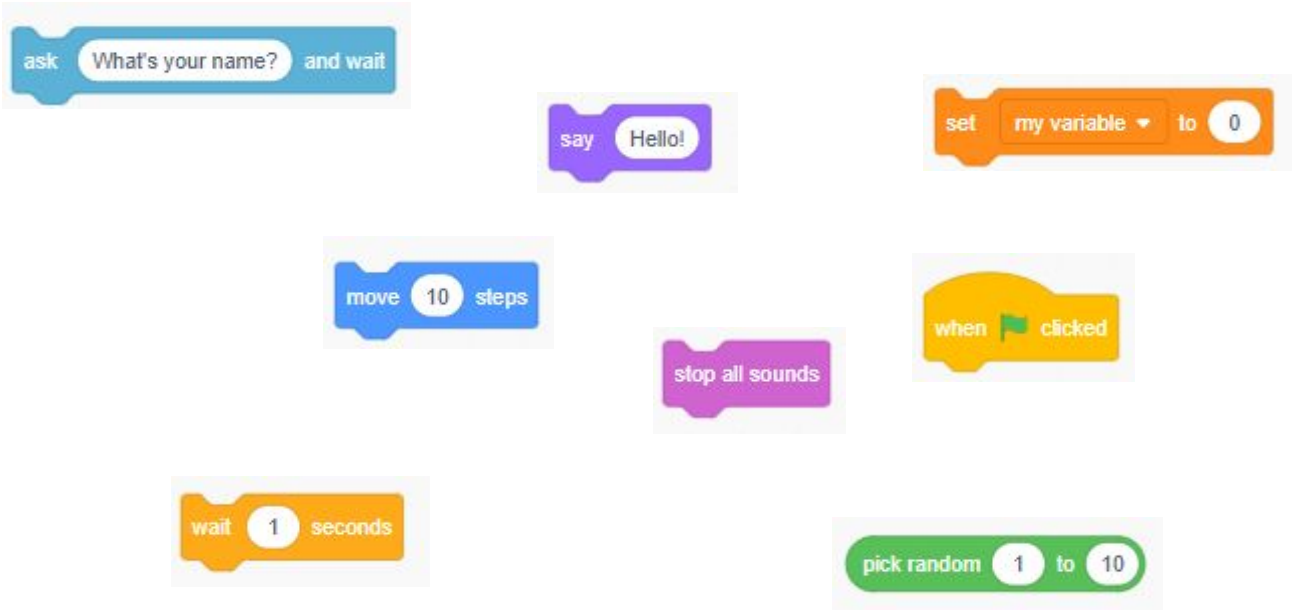
- Block-based visual programming
- For animations, stories, and simple games
- Limited to visual projects and basic logic
- Designed for beginners

Scratch UI


Code
Costumes
Sounds
Motion
Looks
Sound
Events
Control
Sensing
Operators
Variables
My Blocks
Motion
move 10 steps
turn 15 degrees
turn 15 degrees
go to random position
go to x: 0 y: 0
glide 1 secs to random position
glide 1 secs to x: 0 y: 0
point in direction 90
point towards mouse-pointer
change x by 10
set x to 0
change y by 10
set y to 0
if on edge, bounce
Sprite
Sprite1
x
0
y
0
Show
Size
100
Direction
90
Sprite1
Stage
Backdrops
1
Backpack

Scratch UI


Code
Costumes
Sounds
Motion
Looks
Sound
Events
Control
Sensing
Operators
Variables
My Blocks
Motion
move 10 steps
turn 15 degrees
turn 15 degrees
go to random position
go to x: 0 y: 0
glide 1 secs to random position
glide 1 secs to x: 0 y: 0
point in direction 90
point towards mouse-pointer
change x by 10
set x to 0
change y by 10
set y to 0
if on edge, bounce
Backpack
Stage
Sprite
Sprite1
x
0
y
0
Show
Size
100
Direction
90
Sprite1
Stage
Backdrops
1
Block Canvas
Selector


ask What's your name? and wait
say Hello!
set my variable to 0
move 10 steps
when clicked
stop all sounds
wait 1 seconds
pick random 1 to 10

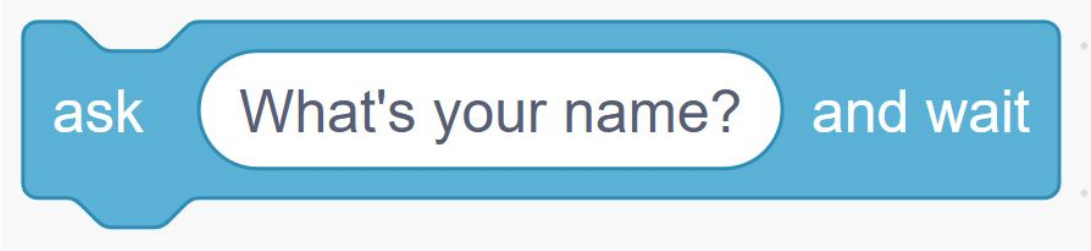
Blocks

Block 4 Major Types

Scratch UI

Input and Display Blocks

```
1  x = input()
```




```
1  print()
```

Python to Blocks

```
1  print("Hello World!")
```

Python to Blocks

```
1  answer = input("Name: ")
2  print(answer)
```

Python to Blocks

```
1  answer = input("What is your name?")
2  print("Hello, ", answer)
```

Python to Blocks

```
1  answer = input("Enter x: ")
2  x = int(answer)
3  answer = input("Enter y: ")
4  y = int(answer)
5  print(x + y)
```

Python to Blocks

```
1  answer = input("Enter number: ")
2  if answer > 100:
3    print("Greater than 100")
4  else:
5    print("Less than 100")
```

Python to Blocks

```
1  answer = ""
2  while answer < 0:
3    print(answer)
4    answer = int(input("Enter number: "))
```

Python Loops

```python
print("Hello!")
print("Hello!")
print("Hello!")
print("Hello!")
print("Hello!")
```

```python
for count in range(5):
  print("Hello!")
```

Python Loops

Two types of loops

- For loops
- While loops

For loops

Know repeat count

```
1  for count in range(5):
2    print("Hello!")
```

For loops

Know repeat count

```
for count in range(5):
  print(count)
```

output

```
0
1
2
3
4
```

For loops

Know repeat count

```
fruits = ["apple","orange"]
for fruit in fruits:
  print(fruit)
```

output

```
apple
orange
```

While loops

Repeat until a false condition

```
x = 0

while x < 5:
  print("Hello!")
  x = x + 1   # x += 1
```

While loops

Repeat until a false condition

```
x = 0

while x < 5:
  print("Hello!")
  x = x + 1   # x += 1
```

While loops

Repeat until a false condition

```python
answer = 0
while answer >= 0:
  answer = int(input("Enter number: "))
  if answer >= 0:
    print("Number not negative.")
  else:
    print("Negative")
```

output

```
Enter number: 10
Number not negative.
Enter number: 5
number not negative.
Enter number: -1
Negative
```

Loops in Scratch

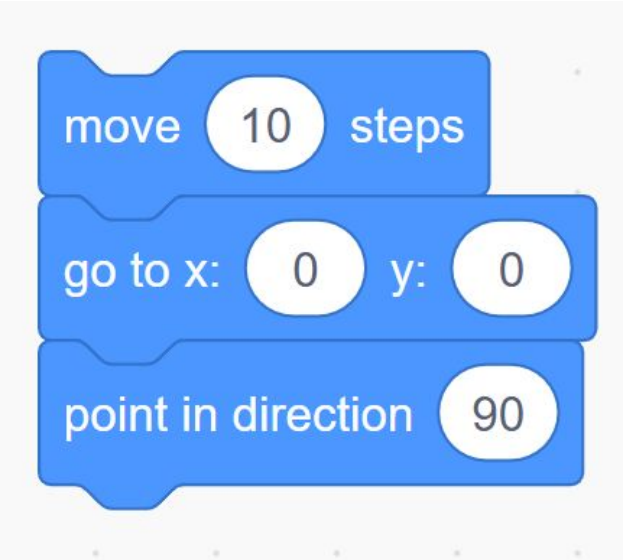
Python to Blocks

```
1  answer = ""
2  while answer < 0:
3    print(answer)
4    answer = int(input("Enter number: "))
```

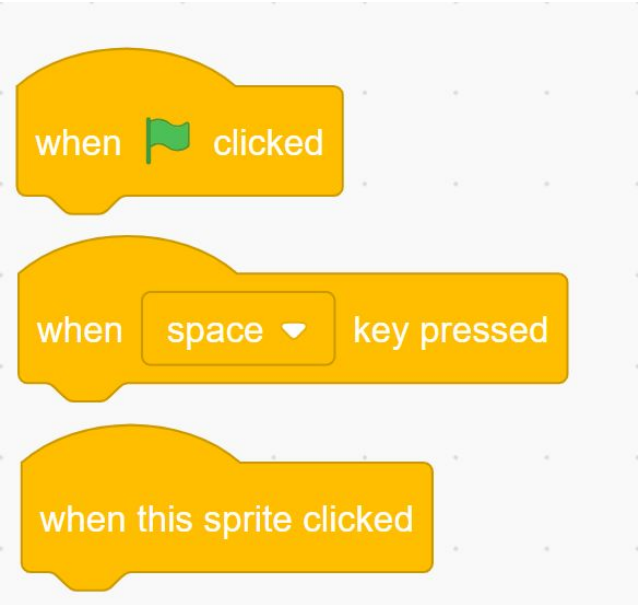
```python
name = input("What is your name? ")
print("Hello", name)

number = int(input("Enter a number: "))
count = 0
for count in range(number):
    count = count + 1

    if count % 2 == 0:
        print(count, "is even")
    else:
```

Motion and Events

Motion

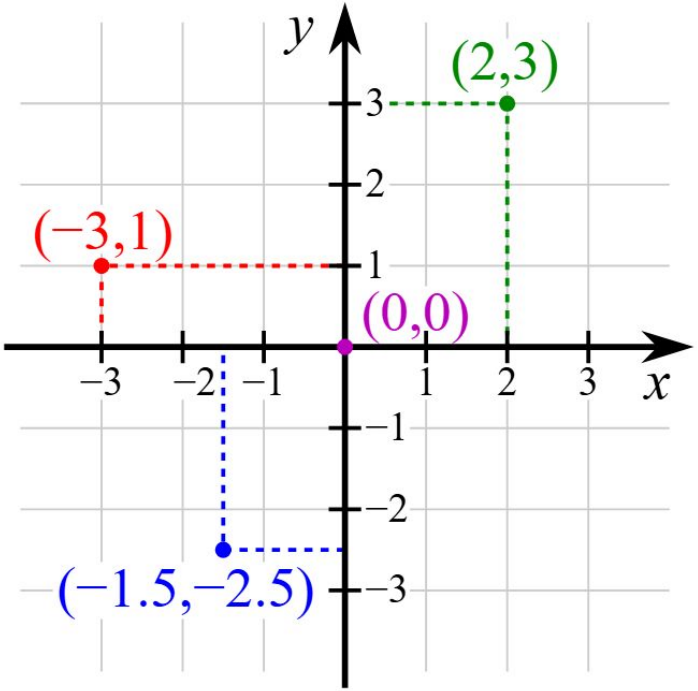


Events



Position / Coordinates

X and Y



Activity

Player Movement System

Instruction 1:

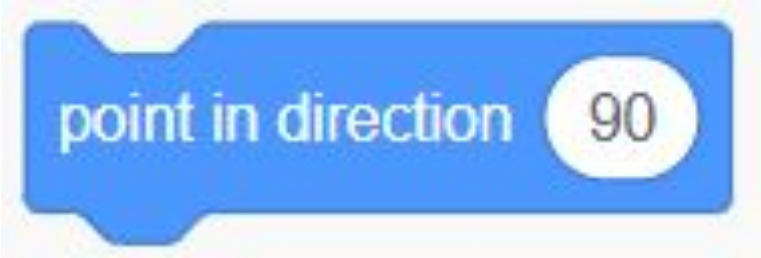
Add left and right movement

Activity

Player Movement System

Instruction 2:

Add Player Rotation

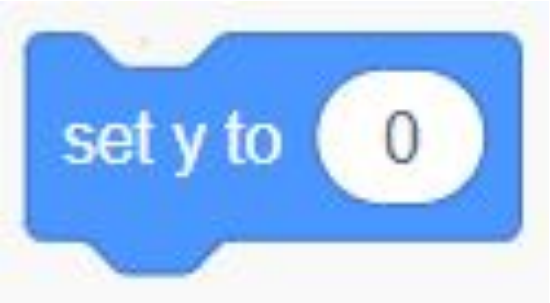


Activity

Player Movement System

Instruction 3 (final):

Using y axis, add up and down movement



Game Design

1. Player Contract

- Unspoken agreement between Game & Player
  - Promise of the game

Game Design

2. Goal & Obstacle

- Every game must have a goal and an obstacle

Structure:

Goal > Obstacle > Action > Result > Repeat

Game Design

3. Feedback

- Visual, Audio, Numerical, Physical
- What happened?
- Why?
- Good or Bad?

Game Design

4. Meaningful Choices

Meaningful when:

- It changes the future
  - It has tradeoffs
  - It’s not obvious

Game Design

5. Escalating Challenge

Games must increase

- Complexity
- Speed
- Pressure
- Skill

Game Design

6. Clear Communication

- Clear communication between game and player

The MDA Framework

MDA stands for

- Mechanics
- Dynamics
- Aesthetics

The MDA Framework

Why is this used?

Hook and Core Loops

Core loop of a game:

Action > Result > Improvement > Repeat

Meta loop of a game:

Complete Core Loop > Unlock Upgrades > Repeat

Hook and Core Loops

Hook:

These are what gets people to start Core Loops

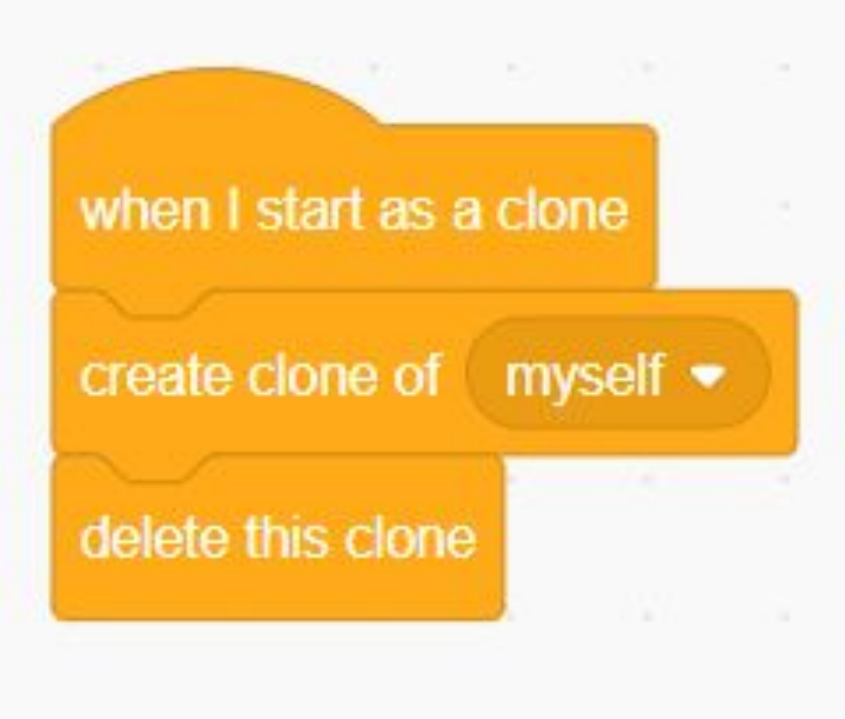
Core Game Mechanics

- Involves Specific Rules
- Movement & Navigation
- Resource
- Conflicts & Resolution
- Progression

Sprites

- Each sprite has their own:
  - Brain
  - Location
  - Looks

Sprite Cloning



Game States

- The current condition or mode the game is in.
  - A game is in one state only.

Menu > Playing > Game Over > Menu